

Islamic University of Technology

Lab 3: Empirical Analysis of Hash Table Collision Resolution

CSE 4810: Algorithm Engineering
1A

ASH, ABT(PT)

August 6, 2026

General Instructions: Empirical Analysis

To observe how different collision resolution strategies and hashing functions affect the operational efficiency and probe counts of a Dictionary ADT.

1. **Implementation:** Implement a Hash Table supporting Insert, Search, and Delete operations using both Linear Probing and Double Hashing according to your assigned flavor.
2. **Experimental Setup:** Run both collision resolution algorithms on your generated dataset. Evaluate performance as the load factor (α) increases from 0.1 to 0.9.
3. **Data Recording:** Save your results to a text file (`data.txt`) in five columns representing Load Factor, Execution Times, and Probe Counts:
`Alpha, Time_LP, Time_DH, Probes_LP, Probes_DH.`
4. **Instrumentation:** Use high-resolution timers to measure bulk execution time. Use a count variable inside your probing loops to measure the average number of probes per operation.
5. **Visualization:** Use `matplotlib` to plot your data (Execution Time vs. α and Probe Count vs. α).
6. **Theoretical Analysis:** Compare your measured probe counts against the theoretical bounds established in Theorems 11.2, 11.6, and 11.8. (Follow only the assigned task in your flavor).

Algorithm Engineering Note

Cache Locality vs. Theoretical Bounds:

For all groups, keep in mind that while theoretical analysis heavily favors Double Hashing for minimizing probe counts, **Linear Probing** may show superior absolute execution speeds in your timing profiles if you use large tables. This is due to hardware cache locality—Linear Probing scans contiguous memory blocks, whereas Double Hashing triggers frequent cache misses.

Flavor 1: The "Prime-Division" Method

Focus: Analyzing the impact of primary clustering in Linear Probing compared against a Double Hashing baseline, and evaluating the efficiency of the Division Method with prime table sizes.

1. Configuration

- **Table Size (m):** Must be a prime number (e.g., $m = 1009$ for the experiment, $m = 13$ for manual tracing).
- **Primary Hash Function (Division Method):** $h(k) = k \bmod m$.
- **Secondary Hash Function (For Double Hashing only):** $h_2(k) = 1 + (k \bmod (m - 1))$.
- **Deletion Logic:** Manual re-hashing of clusters.

2. Unique Experiment: Clustering Observation

- **Implementation Task:** Implement **two distinct** hash tables (or execution modes): one using Linear Probing and one using Double Hashing.
 - **Data Generation:** Use a non-uniform data set (e.g., keys that are arithmetic progressions like 10, 20, 30, ...).
 - **Clustering Task (Linear Probing):** You must explicitly measure the length of the longest "cluster" (contiguous occupied slots) in your Linear Probing implementation as the load factor α increases from 0.5 to 0.9.
 - **Verification & Comparison:** Compare the average number of probes in an **Unsuccessful Search** for both methods against the theoretical limit for Linear Probing: $\frac{1}{1-\alpha}$. Observe and graph how Double Hashing successfully avoids this severe clustering.
-

Part 1: Instrumentation (Time Calculation)

CRITICAL RULE: Never place I/O operations (like `printf`, `cout`, or `print`) inside your timing blocks. Console output is thousands of times slower than basic arithmetic and will completely corrupt your empirical data.

1. C++ (Using `std::chrono`)

Use the provided `HashingLab.cpp` file

3. Python (Using `time.perf_counter()`)

Use the provided `HashingLab.py` file

Part 2: Data Formatting

Your program should run your algorithms inside a loop that gradually increases the load factor α (e.g., $\alpha = 0.1, 0.2, 0.3 \dots 0.9$).

Instead of printing to the console, write the results directly to a text file named `data.txt`. Ensure the data is formatted in 5 distinct columns separated by spaces.

Example `data.txt`:

#Alpha	Time_LP	Time_DH	Probes_LP	Probes_DH
0.1	12	18	1.05	1.05
0.2	25	35	1.12	1.11
0.3	40	52	1.21	1.18
0.4	58	75	1.33	1.25
0.5	80	105	1.50	1.38
0.6	115	140	1.75	1.52
0.7	165	185	2.16	1.71
0.8	250	245	3.00	2.01
0.9	480	320	5.50	2.55

Part 3: Plotting with Python Pyplot

Once you have generated your `data.txt` file, you will use Python and the `matplotlib` library to visualize how your hash tables degrade as the load factor increases.

1. Prerequisites

If your Ubuntu lab PC does not already have `matplotlib` installed, open your terminal and install it using the system package manager:

```
sudo apt update
sudo apt install python3-matplotlib
```

2. The Plotter Script

Use the `plotter.py` file for plotting

3. Running the Script

Run the script from your terminal:

```
python3 plotter.py
```

When the interactive Pyplot window opens, you should clearly observe the exponential spike in probe counts for Linear Probing as α approaches 0.9.